Reguläre Ausdrücke

Carsten Gips (HSBI)

TL;DR
Mit Hilfe von regulären Ausdrücken kann man den Aufbau von Zeichenketten formal beschreiben. Dabei lassen sich direkt die gewünschten Zeichen einsetzen, oder man nutzt Zeichenklassen oder vordefinierte Ausdrücke. Teilausdrücke lassen sich gruppieren und über *Quantifier* kann definiert werden, wie oft ein Teilausdruck vorkommen soll. Die Quantifier sind per Default **greedy** und versuchen so viel wie möglich zu matchen.
Auf der Java-Seite stellt man reguläre Ausdrücke zunächst als `String` dar. Dabei muss darauf geachtet werden, dass ein Backslash im regulären Ausdruck im Java-String geschützt (*escaped*) werden muss, indem jeweils ein weiterer Backslash voran gestellt wird. Mit Hilfe der Klasse `java.util.regex.Pattern` lässt sich daraus ein Objekt mit dem kompilierten regulären Ausdruck erzeugen, was insbesondere bei mehrfacher Verwendung günstiger in der Laufzeit ist. Dem Pattern-Objekt kann man dann den Suchstring übergeben und bekommt ein Objekt der Klasse `java.util.regex.Matcher` (dort sind regulärer Ausdruck/Pattern und der Suchstring kombiniert). Mit den Methoden `Matcher#find` und `Matcher#matches` kann dann geprüft werden, ob das Pattern auf den Suchstring passt: `find` sucht dabei nach dem ersten Vorkommen des Patterns im Suchstring, `match` prüft, ob der gesamte String zum Pattern passt.

Videos

- VL RegExp
- Demo StringSplit
- Demo MatchFind
- Demo Quantifier
- Demo Groups
- Demo Backref

Unterlagen

- PDF-Version

---

1 Suchen in Strings

Gesucht ist ein Programm zum Extrahieren von Telefonnummern aus E-Mails.

=> **Wie geht das?**

Leider gibt es unzählig viele Varianten, wie man eine Telefonnummer (samt Vorwahl und ggf. Ländervorwahl) aufschreiben kann:

```
030 - 123 456 789, 030-123456789, 030/123456789,
+49(30)123456-789, +49 (30) 123 456 - 789, ...
```

2 Definition Regulärer Ausdruck

> Ein **regulärer Ausdruck** ist eine Zeichenkette, die zur Beschreibung von Zeichenketten dient.

2.1 Anwendungen

- Finden von Bestandteilen in Zeichenketten
- Aufteilen von Strings in Tokens
- Validierung von textuellen Eingaben => “Eine Postleitzahl besteht aus 5 Ziffern”
- Compilerbau: Erkennen von Schlüsselwörtern und Strukturen und Syntaxfehlern

3 Einfachste reguläre Ausdrücke

Zeichenkette	Beschreibt
`x`	“x”
`.`	ein beliebiges Zeichen
`\t`	Tabulator
`\n`	Newline
`\r`	Carriage-return
`\\`	Backslash

3.1 Beispiel

- `abc` => “abc”
- `A.B` => “AAB” oder “A2B” oder …
- `a\\bc` => “a\bc”

3.2 Anmerkung

In Java-Strings leitet der Backslash eine zu interpretierende Befehlssequenz ein. Deshalb muss der Backslash i.d.R. geschützt (“escaped”) werden. => Statt “`\n`” müssen Sie im Java-Code “`\\n`” schreiben!

4 Zeichenklassen

Zeichenkette	Beschreibt
`[abc]`	“a” oder “b” oder “c”
`[^abc]`	alles außer “a”, “b” oder “c” (Negation)
`[a-zA-Z]`	alle Zeichen von “a” bis “z” und “A” bis “Z” (Range)
`[a-z&&[def]]`	“d”, “e” oder “f” (Schnitt)
`[a-z&&[^bc]]`	“a” bis “z”, außer “b” und “c”: `[ad-z]` (Subtraktion)
`[a-z&&[^m-p]]`	“a” bis “z”, außer “m” bis “p”: `[a-lq-z]` (Subtraktion)

Zeichenklassen werden über eine Zeichenkette formuliert, die in `[` und `]` eingeschlossen wird. Dabei werden alle Zeichen aufgezählt, die in dieser Zeichenklasse enthalten sein sollen. Die Zeichenklasse verhält sich von außen betrachtet wie ein beliebiges Zeichen aus der Menge der aufgezählten Zeichen.

Beispiel: `[abc]` meint ein “a” oder “b” oder “c” …

Wenn dem ersten Zeichen der so geformten Zeichenklasse ein ^ vorangestellt wird, sind alle Zeichen *außer* den in der Zeichenklasse bezeichneten Zeichen gemeint (Negation). In der Tabelle oben (erste Zeile) könnte man dem `abc` noch ein ^ voranstellen und hätte dann *alle* Zeichen *außer* “a”, “b” und “c”.

Für den Schnitt kann als zweite Zeichenklasse eine Negation verwendet werden, damit würde eine Subtraktion erreicht werden: Alle Zeichen in der vorderen Zeichenklasse abzüglich der Zeichen in der zweiten Zeichenklasse. In der Tabelle oben (vierte Zeile) würde man dem `def` noch ein ^ voranstellen und hätte dann die Zeichen “a” bis “z” *ohne* “d”, “e” und “f”.

Anmerkung: Das Minus-Zeichen hat in der Zeichenklasse eine besondere Bedeutung (es bildet einen Range). Deshalb muss es escaped werden, wenn es sich selbst darstellen soll.

4.1 Beispiel

- `[abc]` => “a” oder “b” oder “c”
- `[a-c]` => “a” oder “b” oder “c”
- `[a-c][a-c]` => “aa”, “ab”, “ac”, “ba”, “bb”, “bc”, “ca”, “cb” oder “cc”
- `A[a-c]` => “Aa”, “Ab” oder “Ac”

5 Vordefinierte Ausdrücke

Zeichenkette	Beschreibt
^	Zeilenanfang
`$`	Zeilenende

Zeichenkette	Beschreibt
`\d`	eine Ziffer: `[0-9]`
`\w`	beliebiges Wortzeichen: `[a-zA-Z_0-9]`
`\s`	Whitespace (Leerzeichen, Tabulator, Newline)
`\D`	jedes Zeichen außer Ziffern: `[^0-9]`
`\W`	jedes Zeichen außer Wortzeichen: `[^\w]`
`\S`	jedes Zeichen außer Whitespaces: `[^\s]`

5.1 Beispiel

- `\d\d\d\d\d` => "12345"
- `\w\wA` => "aaA", "a0A", "a_A", ...

6 Nutzung in Java

- `java.lang.String`:

```
public String[] split(String regex)
public boolean matches(String regex)
```

Demo: regexp.StringSplit

- `java.util.regex.Pattern`:

```
public static Pattern compile(String regex)
public Matcher matcher(CharSequence input)
```

  - Schritt 1: Ein Pattern compilieren (erzeugen) mit `Pattern#compile` => liefert ein Pattern-Objekt für den regulären Ausdruck zurück
  - Schritt 2: Dem Pattern-Objekt den zu untersuchenden Zeichenstrom übergeben mit `Pattern#matcher` => liefert ein Matcher-Objekt zurück, darin gebunden: Pattern (regulärer Ausdruck) und die zu untersuchende Zeichenkette

- `java.util.regex.Matcher`:

```
public boolean find()
public boolean matches()
public int groupCount()
public String group(int group)
```

  - Schritt 3: Mit dem Matcher-Objekt kann man die Ergebnisse der Anwendung des regulären Ausdrucks auf eine Zeichenkette auswerten

Bedeutung der unterschiedlichen Methoden siehe folgende Folien

`Matcher#group`: Liefert die Sub-Sequenz des Suchstrings zurück, die erfolgreich gematcht wurde (siehe unten "Fangende Gruppierungen")

Hinweis:

In Java-Strings leitet der Backslash eine zu interpretierende Befehlssequenz ein. Deshalb muss der Backslash i.d.R. extra geschützt ("escaped") werden.

=> Statt "`\n`" (regulärer Ausdruck) müssen Sie im Java-String "`\\n`" schreiben!

=> Statt "`a\\bc`" (regulärer Ausdruck, passt auf die Zeichenkette "a\bc") müssen Sie im Java-String "`a\\\\bc`" schreiben!

Demo: regexp.MatchFind

7 Unterschied zw. Finden und Matchen

- `Matcher#find`:

  Regulärer Ausdruck muss im Suchstring **enthalten** sein.
  => Suche nach **erstem Vorkommen**

- `Matcher#matches`:

  Regulärer Ausdruck muss auf **kompletten** Suchstring passen.

7.1 Beispiel

- Regulärer Ausdruck: `abc`, Suchstring: "blah blah abc blub"
  - `Matcher#find`: erfolgreich
  - `Matcher#matches`: kein Match - Suchstring entspricht nicht dem Muster

8 Quantifizierung

Zeichenkette	Beschreibt
`X?`	ein oder kein "X"
`X*`	beliebig viele "X" (inkl. kein "X")
`X+`	mindestens ein "X", ansonsten beliebig viele "X"
`X{n}`	exakt n Vorkommen von "X"

Zeichenkette	Beschreibt
`X{n,}`	mindestens n Vorkommen von “X”
`X{n,m}`	zwischen n und m Vorkommen von “X”

8.1 Beispiel

- `\d{5}` => “12345”
- `-?\d+\.\d*` => ???

9 Interessante Effekte

```
Pattern p = Pattern.compile("A.*A");
Matcher m = p.matcher("A 12 A 45 A");

if (m.matches())
    String result = m.group(); // ???
```

Demo: regexp.Quantifier

`Matcher#group` liefert die Inputsequenz, auf die der Matcher angesprochen hat. Mit `Matcher#start` und `Matcher#end` kann man sich die Indizes des ersten und letzten Zeichens des Matches im Eingabezeichenstrom geben lassen. D.h. für einen Matcher `m` und eine Eingabezeichenkette `s` ist `m.group()` und `s.substring(m.start(), m.end())` äquivalent.

Da bei `Matcher#matches` das Pattern immer auf den gesamten Suchstring passen muss, verwundert das Ergebnis für `Matcher#group` nicht. Bei `Matcher#find` wird im Beispiel allerdings ebenfalls der gesamte Suchstring “gefunden” … Dies liegt am “*greedy*” Verhalten der Quantifizierer.

10 Nicht gierige Quantifizierung mit “?”

Zeichenkette	Beschreibt
`X*?`	non-greedy Variante von `X*`
`X+?`	non-greedy Variante von `X+`

10.1 Beispiel

- Suchstring “A 12 A 45 A”:

- `A.*A` findet/passt auf “A 12 A 45 A”

  normale **greedy** Variante
- `A.*?A`
  * findet “A 12 A”
  * passt auf “A 12 A 45 A” (!)

  non-greedy Variante der Quantifizierung; `Matcher#matches` muss trotzdem auf den gesamten Suchstring passen!

11 (Fangende) Gruppierungen

`Studi{2}` passt nicht auf “StudiStudi” (!)

Quantifizierung bezieht sich auf das direkt davor stehende Zeichen. Ggf. Gruppierungen durch Klammern verwenden!

Zeichenkette	Beschreibt
`X|Y`	X oder Y
`(C)`	Gruppierung

11.1 Beispiel

- `(A)(B(C))`
  - Gruppe 0: `ABC`
  - Gruppe 1: `A`
  - Gruppe 2: `BC`
  - Gruppe 3: `C`

Die Gruppen heißen auch “fangende” Gruppen (engl.: *“capturing groups”*).

Damit erreicht man eine Segmentierung des gesamten regulären Ausdrucks, der in seiner Wirkung aber nicht durch die Gruppierungen geändert wird. Durch die Gruppierungen von Teilen des regulären Ausdrucks erhält man die Möglichkeit, auf die entsprechenden Teil-Matches (der Unterausdrücke der einzelnen Gruppen) zuzugreifen:

- `Matcher#groupCount`: Anzahl der “fangenden” Gruppen im regulären Ausdruck
- `Matcher#group(i)`: Liefert die Subsequenz der Eingabezeichenkette zurück, auf die die jeweilige Gruppe gepasst hat. Dabei wird von links nach rechts durchgezählt, beginnend bei 1(!).

  Konvention: Gruppe 0 ist das gesamte Pattern, d.h. `m.group(0)== m.group();` ...

Hinweis: Damit der Zugriff auf die Gruppen klappt, muss auch erst ein Match gemacht werden, d.h. das Erzeugen des Matcher-Objekts reicht noch nicht, sondern es muss auch noch ein `matcher.find()` oder `matcher.matches()` ausgeführt werden. Danach kann man bei Vorliegen eines Matches auf die Gruppen zugreifen.

`(Studi){2}` => "StudiStudi"

Demo: regexp.Groups

12 Gruppen und Backreferences

Matche zwei Ziffern, gefolgt von den selben zwei Ziffern

`(\d\d)\1`

- Verweis auf bereits gematchte Gruppen: `\num`

  `num` Nummer der Gruppe (1 … 9)

  => Verweist nicht auf regulären Ausdruck, sondern auf jeweiligen Match!

  Anmerkung: Laut Literatur/Doku nur 1 … 9, in Praxis geht auch mehr per Backreference …

- Benennung der Gruppe: `(?<name>X)`

  `X` ist regulärer Ausdruck für Gruppe, spitze Klammern wichtig

  => Backreference: `\k<name>`

Demo: regexp.Backref

13 Beispiel Gruppen und Backreferences

Regulärer Ausdruck: Namen einer Person matchen, wenn Vor- und Nachname identisch sind.

Lösung: `([A-Z][a-zA-Z]*)\s\1`

14 Umlaute und reguläre Ausdrücke

- Keine vordefinierte Abkürzung für Umlaute (wie etwa `\d`)
- Umlaute nicht in `[a-z]` enthalten, aber in `[a-ü]`

```
"helloüA".matches(".*?[ü]A");
"azäöüß".matches("[a-ä]");
"azäöüß".matches("[a-ö]");
"azäöüß".matches("[a-ü]");
"azäöüß".matches("[a-ß]");
```

- Strings sind Unicode-Zeichenketten

  => Nutzung der passenden Unicode Escape Sequence `\uFFFF`

```
System.out.println("\u0041 :: A");
System.out.println("helloüA".matches(".*?A"));
System.out.println("helloüA".matches(".*?\u0041"));
System.out.println("helloü\u0041".matches(".*?A"));
```

- RegExp vordefinieren und mit Variablen zusammenbauen ala Perl nicht möglich => Umweg String-Repräsentation

15 Wrap-Up

- RegExp: Zeichenketten, die andere Zeichenketten beschreiben
- `java.util.regex.Pattern` und `java.util.regex.Matcher`
- Unterschied zwischen `Matcher#find` und `Matcher#matches`!
- Quantifizierung ist möglich, aber **greedy** (Default)

16 Zum Nachlesen

- Oracle Corporation (2024)

Lernziele

- k1: Ich kenne die wichtigsten Methoden von java.util.regex.Pattern und java.util.regex.Matcher
- k2: Ich kann den Unterschied zwischen Matcher#find und Matcher#matches erklären
- k2: Ich kann zwischen greedy und non-greedy Verhalten bei regulären Ausdrücken unterscheiden
- k3: Ich kann einfache reguläre Ausdrücke bilden
- k3: Ich kann Zeichenklassen und deren Negation einsetzen
- k3: Ich kann die vordefinierten regulären Ausdrücke einsetzen
- k3: Ich kann Quantifizierer gezielt einsetzen
- k3: Ich kann komplexe Ausdrücke (u.a. mit Gruppen) konstruieren

Challenges

Schreiben Sie eine Methode, die mit Hilfe von regulären Ausdrücken überprüft, ob der eingegebene String eine nach dem folgenden Schema gebildete EMail-Adresse ist:

`name@firma.domain`

Dabei sollen folgende Regeln gelten:

- Die Bestandteile `name` und `firma` können aus Buchstaben, Ziffern, Unter- und Bindestrichen bestehen.
- Der Bestandteil `name` muss mindestens ein Zeichen lang sein.
- Der Bestandteil `firma` kann entfallen, dann entfällt auch der nachfolgende Punkt (`.`) und der Teil `domain` folgt direkt auf das `@`-Zeichen.
- Der Bestandteil `domain` besteht aus 2 oder 3 Kleinbuchstaben.

Hinweis: Sie dürfen keinen Oder-Operator verwenden.

Quellen

Oracle Corporation. 2024. „The Java Tutorials“. 2024. https://docs.oracle.com/javase/tutorial/.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** f0e0ac2 (lecture: fix description of regexp examples (Regexp), 2025-05-25)